STAFF REVIEW · H446 023 source models

Separate from the eight-page pupil workbook. This copy is in a public repository and is NOT access controlled. Check technical correctness before use.

Printed question mapping: Booklet Q1 = original online Q1; Booklet Q2 = original online Q4

Do Now — source answers

1. Smaller values go left and larger values go right.

2. null / no child.

3. Two keys map to the same hash location.

4. 2.

I Do — source worked steps

1. 12 — CODE:
table[2,0] = 12
count[2] = 1

2. 17 — CODE:
table[2,1] = 17
count[2] = 2

3. 22 — CODE:
table[2,2] = 22
count[2] = 3

4. 17 — CODE:
hash → bucket 2
scan row 2
17 found at column 1

We Do — guided CHECK

1. Complete the same calculation
CODE:
37 MOD 10 = 7

2. Fill the same code
CODE:
int bucket = key % ?;
table[bucket, ?] = key;
count[bucket]++;
CODE:
int bucket = key % 5;
table[bucket, count[bucket]] = key;
count[bucket]++;

3. Complete the bucket
CODE:

bucket 2:
column 0 = 12
column 1 = ?
CODE:
bucket 2:
column 0 = 12
column 1 = 17

4. Complete the shift
CODE:
table[b,1] = table[b,?];
count[b]--;
CODE:
table[b,1] = table[b,2];
count[b]--;

You Do — source mark schemes

1. Create a table with 5 rows (buckets) and capacity 3 per bucket. Counts begin at zero.
CODE:
int[,] buckets = new int[5,3];
int[] counts = new int[5];
Use counts[b] as the next free column in bucket b.

2. Hash each key using key MOD 5.
CODE:
int[] keys = {12,17,22,9};
foreach (int key in keys)
{
int b = key % 5;
buckets[b,counts[b]] = key;
counts[b]++;
}
12 MOD 5 = 2; 17 MOD 5 = 2; 22 MOD 5 = 2; 9 MOD 5 = 4.
CODE:
Bucket 2: [12, 17, 22]
Bucket 4: [9, -, -]
Collision resolution: keys sharing a hash occupy successive columns.

3. Hash 17 first, then search only bucket 2.
CODE:
int key = 17;
int b = key % 5;
bool found = false;
for (int i = 0; i < counts[b]; i++)
{
if (buckets[b,i] == key)
found = true;
}

Console.WriteLine(found); // True

4. Find the key in its bucket; shift subsequent entries to close the gap.
CODE:

```
int key = 17;
int b = key % 5;
for (int i = 0; i < counts[b]; i++)
{
if (buckets[b,i] == key)
{
for (int j = i; j < counts[b]-1; j++)
buckets[b,j] = buckets[b,j+1];
counts[b]--;
break;
}
}
```

Bucket 2 becomes [12, 22, —], count = 2. Searching for 22 still succeeds.

Exit Ticket — source answer

1. True. Different keys can hash to the same bucket. This is called a collision.